

### 2.1.3 Pertanyaan

1. Lakukan perbaikan pada kode program, sehingga keluaran yang dihasilkan sama dengan verifikasi hasil percobaan! Bagian mana yang perlu diperbaiki?

```
// PERBAIKAN OUTPUT
public void print() {
    if (isEmpty()) {
        System.out.println(x: "Stack kosong! Tidak ada tugas yang dikumpulkan.");
    } else {
        for (int i = top; i >= 0; i--) {
            System.out.println(stack[i].nama + "\t"
                               + stack[i].nim + "\t"
                               + stack[i].kelas);
        }
        System.out.println(x: "");
    }
}
```

2. Berapa banyak data tugas mahasiswa yang dapat ditampung di dalam Stack? Tunjukkan potongan kode programnya!

Sebanyak nilai parameter saat membuat objek stack.

3. Mengapa perlu pengecekan kondisi !isFull() pada method push? Kalau kondisi if-else tersebut dihapus, apa dampaknya?

Untuk mencegah stack overflow.

4. Modifikasi kode program pada class MahasiswaDemo dan StackTugasMahasiswa sehingga pengguna juga dapat melihat mahasiswa yang pertama kali mengumpulkan tugas melalui operasi lihat tugas terbawah!

```
// TAMBAHAN NO 4
// Method untuk melihat mahasiswa pertama (terbawah)
public Mahasiswa17 peekBottom() {
    if (!isEmpty()) {
        return stack[0];
    } else {
        System.out.println(x: "Stack kosong! Tidak ada tugas.");
        return null;
    }
}
```

```
//TAMBAHAN NO 4
case 5:
    Mahasiswa17 bawah = stack.peekBottom();
    if (bawah != null) {
        System.out.println("Tugas pertama dikumpulkan oleh " + bawah.nama);
    }
    break;
// AKHIR TAMBAHAN
default:
    System.out.println(x: "Pilihan tidak valid.");
```

5. Tambahkan method untuk dapat menghitung berapa banyak tugas yang sudah dikumpulkan saat ini, serta tambahkan operasi menunya!

```
// Perubahan No 5
public int jumlahTugas() {
    return top + 1;
}
```

```
// Perubahan No 5
case 6:
    System.out.println("Jumlah tugas yang sudah dikumpulkan: " + stack.jumlahTugas());
    break;

default:
    System.out.println(x: "Pilihan tidak valid.");
}
```

### 2.2.3 Pertanyaan

1. Jelaskan alur kerja dari method konversiDesimalKeBiner!

Method konversiDesimalKeBiner bekerja dengan cara membagi angka desimal dengan 2 terus-menerus. Sisa hasil pembagian disimpan ke dalam stack menggunakan push(). Setelah itu, data diambil kembali dengan pop() sehingga menghasilkan urutan bilangan biner yang benar.

2. Pada method konversiDesimalKeBiner, ubah kondisi perulangan menjadi while (kode != 0), bagaimana hasilnya? Jelaskan alasannya!

hasilnya tetap benar dan program tetap berjalan normal. Karena perulangan akan terus berjalan selama nilai kode belum 0. Setelah kode menjadi 0, proses berhenti.